

10.6 Exercises

Reinforcement

- R-10.1 What is the worst-case running time for inserting n key-value pairs into an initially empty map M that is implemented with the `UnsortedTableMap` class?
- R-10.2 Reimplement the `UnsortedTableMap` class using the `PositionalList` class from Section 7.3 rather than an `ArrayList`.
- R-10.3 The use of null values in a map is problematic, as there is then no way to differentiate whether a null value returned by the call `get(k)` represents the legitimate value of an entry (k, null) , or designates that key k was not found. The `java.util.Map` interface includes a boolean method, `containsKey(k)`, that resolves any such ambiguity. Implement such a method for the `UnsortedTableMap` class.
- R-10.4 Which of the hash table collision-handling schemes could tolerate a load factor above 1 and which could not?
- R-10.5 What would be a good hash code for a vehicle identification number that is a string of numbers and letters of the form “9X9XX99X9XX999999,” where a “9” represents a digit and an “X” represents a letter?
- R-10.6 Draw the 11-entry hash table that results from using the hash function, $h(i) = (3i + 5) \bmod 11$, to hash the keys 12, 44, 13, 88, 23, 94, 11, 39, 20, 16, and 5, assuming collisions are handled by chaining.
- R-10.7 What is the result of the previous exercise, assuming collisions are handled by linear probing?
- R-10.8 Show the result of Exercise R-10.6, assuming collisions are handled by quadratic probing, up to the point where the method fails.
- R-10.9 What is the result of Exercise R-10.6 when collisions are handled by double hashing using the secondary hash function $h'(k) = 7 - (k \bmod 7)$?
- R-10.10 What is the worst-case time for putting n entries in an initially empty hash table, with collisions resolved by chaining? What is the best case?
- R-10.11 Show the result of rehashing the hash table shown in Figure 10.6 into a table of size 19 using the new hash function $h(k) = 3k \bmod 17$.
- R-10.12 Modify the `Pair` class from Code Fragment 2.17 on page 92 so that it provides a natural definition for both the `equals()` and `hashCode()` methods.
- R-10.13 Consider lines 31–33 of Code Fragment 10.8 in our implementation of the class `ChainHashMap`. We use the difference in the size of a secondary bucket before and after a call to `bucket.remove(k)` to update the variable `n`. If we replace those three lines with the following, does the class behave properly? Explain.

```

V answer = bucket.remove(k);
if (answer != null)           // value of removed entry
    n--;                     // size has decreased

```

- R-10.14 Our `AbstractHashMap` class maintains a load factor $\lambda \leq 0.5$. Reimplement that class to allow the user to specify the maximum load, and adjust the concrete subclasses accordingly.
- R-10.15 Give a pseudocode description of an insertion into a hash table that uses quadratic probing to resolve collisions, assuming we also use the trick of replacing deleted entries with a special “available” object.
- R-10.16 Modify our `ProbeHashMap` to use quadratic probing.
- R-10.17 Explain why a hash table is not suited to implement a sorted map.
- R-10.18 What is the worst-case asymptotic running time for performing n deletions from a `SortedTableMap` instance that initially contains $2n$ entries?
- R-10.19 Implement the `containKey( $k$ )` method, as described in Exercise R-10.3, for the `SortedTableClass`.
- R-10.20 Describe how a sorted list implemented as a doubly linked list could be used to implement the sorted map ADT.
- R-10.21 Consider the following variant of the `findIndex` method of the `SortedTableMap` class, originally given in Code Fragment 10.11:
- ```

1 private int findIndex(K key, int low, int high) {
2 if (high < low) return high + 1;
3 int mid = (low + high) / 2;
4 if (compare(key, table.get(mid)) < 0)
5 return findIndex(key, low, mid - 1);
6 else
7 return findIndex(key, mid + 1, high);
8 }
```
- Does this always produce the same result as the original version? Justify your answer.
- R-10.22 What is the expected running time of the methods for maintaining a maxima set if we insert  $n$  pairs such that each pair has lower cost and performance than one before it? What is contained in the sorted map at the end of this series of operations? What if each pair had a lower cost and higher performance than the one before it?
- R-10.23 Draw the result after performing the following series of operations on the skip list shown in Figure 10.13: `remove(38)`, `put(48,  $x$ )`, `put(24,  $y$ )`, `remove(55)`. Use an actual coin flip to generate random bits as needed (and report your sequence of flips).
- R-10.24 Give a pseudocode description of the `remove map` operation for a skip list.
- R-10.25 Give a description, in pseudocode, for implementing the `removeAll` method for the set ADT, using only the other fundamental methods of the set.
- R-10.26 Give a description, in pseudocode, for implementing the `retainAll` method for the set ADT, using only the other fundamental methods of the set.

- R-10.27 If we let  $n$  denote the size of set  $S$ , and  $m$  denote the size of set  $T$ , what would be the running time of the operation  $S.addAll(T)$ , as implemented on page 446, if both sets were implemented as skip lists?
- R-10.28 If we let  $n$  denote the size of set  $S$ , and  $m$  denote the size of set  $T$ , what would be the running time of the operation  $S.addAll(T)$ , as implemented on page 446, if both sets were implemented using hashing?
- R-10.29 If we let  $n$  denote the size of set  $S$ , and  $m$  denote the size of set  $T$ , what would be the running time of the operation  $S.removeAll(T)$  when both sets are implemented using hashing?
- R-10.30 If we let  $n$  denote the size of set  $S$ , and  $m$  denote the size of set  $T$ , what would be the running time of the operation  $S.retainAll(T)$  when both sets are implemented using hashing?
- R-10.31 What abstraction would you use to manage a database of friends' birthdays in order to support efficient queries such as "find all friends whose birthday is today" and "find the friend who will be the next to celebrate a birthday"?

## Creativity

- C-10.32 For an ideal compression function, the capacity of the bucket array for a hash table should be a prime number. Therefore, we consider the problem of locating a prime number in a range  $[M, 2M]$ . Implement a method for finding such a prime by using the *sieve algorithm*. In this algorithm, we allocate a  $2M$  cell boolean array  $A$ , such that cell  $i$  is associated with the integer  $i$ . We then initialize the array cells to all be "true" and we "mark off" all the cells that are multiples of 2, 3, 5, 7, and so on. This process can stop after it reaches a number larger than  $\sqrt{2M}$ . (Hint: Consider a bootstrapping method for finding the primes up to  $\sqrt{2M}$ .)
- C-10.33 Consider the goal of adding entry  $(k, v)$  to a map only if there does not yet exist some other entry with key  $k$ . For a map  $M$  (without null values), this might be accomplished as follows.

```
if (M.get(k) == null)
 M.put(k, v);
```

While this accomplishes the goal, its efficiency is less than ideal, as time will be spent on the failed search during the get call, and again during the put call (which always begins by trying to locate an existing entry with the given key). To avoid this inefficiency, some map implementations support a custom method `putIfAbsent(k, v)` that accomplishes this goal. Given such an implementation of `putIfAbsent` for the `UnsortedTableMap` class.

- C-10.34 Repeat Exercise C-10.33 for the `ChainHashMap` class.
- C-10.35 Repeat Exercise C-10.33 for the `ProbeHashMap` class.
- C-10.36 Describe how to redesign the `AbstractHashMap` framework to include support for a method, `containsKey`, as described in Exercise R-10.3.

- C-10.37 Modify the ChainHashMap class in accordance with your design for the previous exercise.
- C-10.38 Modify the ProbeHashMap class in accordance with Exercise C-10.36.
- C-10.39 Redesign the AbstractHashMap class so that it halves the capacity of the table if the load factor falls below 0.25. Your solution must not involve any changes to the concrete ProbeHashMap and ChainHashMap classes.
- C-10.40 The `java.util.HashMap` class uses separate chaining, but without any explicit secondary structures. The table is an array of entries, and each entry has an additional next field that can reference another entry in that bucket. In this way, the entry instances can be threaded as a singly linked list. Reimplement our ChainHashMap class using such an approach.
- C-10.41 Describe how to perform a removal from a hash table that uses linear probing to resolve collisions where we do not use a special marker to represent deleted elements. That is, we must rearrange the contents so that it appears that the removed entry was never inserted in the first place.
- C-10.42 The quadratic probing strategy has a clustering problem related to the way it looks for open slots. Namely, when a collision occurs at bucket  $h(k)$ , it checks buckets  $A[(h(k) + i^2) \bmod N]$ , for  $i = 1, 2, \dots, N - 1$ .
- Show that  $i^2 \bmod N$  will assume at most  $(N + 1)/2$  distinct values, for  $N$  prime, as  $i$  ranges from 1 to  $N - 1$ . As a part of this justification, note that  $i^2 \bmod N = (N - i)^2 \bmod N$  for all  $i$ .
  - A better strategy is to choose a prime  $N$  such that  $N \bmod 4 = 3$  and then to check the buckets  $A[(h(k) \pm i^2) \bmod N]$  as  $i$  ranges from 1 to  $(N - 1)/2$ , alternating between plus and minus. Show that this alternate version is guaranteed to check every bucket in  $A$ .
- C-10.43 Redesign our ProbeHashMap class so that the sequence of secondary probes for collision resolution can be more easily customized. Demonstrate your new design by providing separate concrete subclasses for linear probing and quadratic probing.
- C-10.44 The `java.util.LinkedHashMap` class is a subclass of the standard HashMap class that retains the expected  $O(1)$  performance for the primary map operations while guaranteeing that iterations report entries of the map according to first-in, first-out (FIFO) principle. That is, the key that has been in the map the longest is reported first. (The order is unaffected when the value for an existing key is changed.) Describe an algorithmic approach for achieving such performance.
- C-10.45 Develop a location-aware version of the UnsortedTableMap class so that an operation `remove( $e$ )` for existing Entry  $e$  can be implemented in  $O(1)$  time.
- C-10.46 Repeat the previous exercise for the ProbeHashMap class.
- C-10.47 Repeat Exercise C-10.45 for the ChainHashMap class.

- C-10.48 Although keys in a map are distinct, the binary search algorithm can be applied in a more general setting in which an array stores possibly duplicative elements in nondecreasing order. Consider the goal of identifying the index of the *leftmost* element with key greater than or equal to given  $k$ . Does the `findIndex` method as given in Code Fragment 10.11 guarantee such a result? Does the `findIndex` method as given in Exercise R-10.21 guarantee such a result? Justify your answers.
- C-10.49 Suppose we are given two sorted search tables  $S$  and  $T$ , each with  $n$  entries (with  $S$  and  $T$  being implemented with arrays). Describe an  $O(\log^2 n)$ -time algorithm for finding the  $k^{\text{th}}$  smallest key in the union of the keys from  $S$  and  $T$  (assuming no duplicates).
- C-10.50 Give an  $O(\log n)$ -time solution for the previous problem.
- C-10.51 Give an alternative implementation of the `SortedTableMap`'s `entrySet` method that creates a *lazy iterator* rather than a snapshot. (See Section 7.4.2 for discussion of iterators.)
- C-10.52 Repeat the previous exercise for the `ChainHashMap` class.
- C-10.53 Repeat Exercise C-10.51 for the `ProbeHashMap` class.
- C-10.54 Given a database  $D$  of  $n$  cost-performance pairs  $(c, p)$ , describe an algorithm for finding the maxima pairs of  $C$  in  $O(n \log n)$  time.
- C-10.55 Show that the methods `above( $p$ )` and `before( $p$ )` are not actually needed to efficiently implement a map using a skip list. That is, we can implement insertions and deletions in a skip list using a strictly top-down, scan-forward approach, without ever using the `above` or `before` methods. (Hint: In the insertion algorithm, first repeatedly flip the coin to determine the level where you should start inserting the new entry.)
- C-10.56 Describe how to modify the skip-list data structure to support the method `median()`, which returns the position of the element in the “bottom” list  $S_0$  at index  $\lfloor n/2 \rfloor$ . Show that your implementation of this method runs in  $O(\log n)$  expected time.
- C-10.57 Describe how to modify a skip-list representation so that index-based operations, such as retrieving the entry at index  $j$ , can be performed in  $O(\log n)$  expected time.
- C-10.58 Suppose that each row of an  $n \times n$  array  $A$  consists of 1's and 0's such that, in any row of  $A$ , all the 1's come before any 0's in that row. Assuming  $A$  is already in memory, describe a method running in  $O(n \log n)$  time (not  $O(n^2)$  time) for counting the number of 1's in  $A$ .
- C-10.59 Give a concrete implementation of the `retainAll` method for the set ADT, using only the other fundamental methods of the set. You are to assume that the underlying set implementation uses *fail-fast iterators* (see Section 7.4.2).

- C-10.60 Consider sets whose elements are integers in the range  $[0, N - 1]$ . A popular scheme for representing a set  $A$  of this type is by means of a boolean array,  $B$ , where we say that  $x$  is in  $A$  if and only if  $B[x] = \text{true}$ . Since each cell of  $B$  can be represented with a single bit,  $B$  is sometimes referred to as a **bit vector**. Describe and analyze efficient algorithms for performing the methods of the set ADT assuming this representation.
- C-10.61 An **inverted file** is a critical data structure for implementing applications such as an index of a book or a search engine. Given a document  $D$ , which can be viewed as an unordered, numbered list of words, an inverted file is an ordered list of words,  $L$ , such that, for each word  $w$  in  $L$ , we store the indices of the places in  $D$  where  $w$  appears. Design an efficient algorithm for constructing  $L$  from  $D$ .
- C-10.62 The operation  $\text{get}(k)$  for our multimap ADT is responsible for returning a collection of *all* values currently associated with key  $k$ . Design a variation of binary search for performing this operation on a sorted search table that includes duplicates, and show that it runs in time  $O(s + \log n)$ , where  $n$  is the number of elements in the dictionary and  $s$  is the number of entries with given key  $k$ .
- C-10.63 Describe an efficient multimap structure for storing  $n$  entries that have an associated set of  $r < n$  keys that come from a total order. That is, the set of keys is smaller than the number of entries. Your structure should perform operation  $\text{getAll}$  in  $O(\log r + s)$  expected time, where  $s$  is the number of entries returned, operation  $\text{entrySet}()$  in  $O(n)$  time, and the remaining operations of the multimap ADT in  $O(\log r)$  expected time.
- C-10.64 Describe an efficient multimap structure for storing  $n$  entries whose  $r < n$  keys have distinct hash codes. Your structure should perform operation  $\text{getAll}$  in  $O(1 + s)$  expected time, where  $s$  is the number of entries returned, operation  $\text{entrySet}()$  in  $O(n)$  time, and the remaining operations of the multimap ADT in  $O(1)$  expected time.

---

## Projects

- P-10.65 An interesting strategy for hashing with open addressing is known as **cuckoo hashing**. Two independent hash functions are computed for each key, and an element is always stored in one of the two cells indicated by those hash functions. When a new element is inserted, if either of those two cells is available, it is placed there. Otherwise, it is placed into one of its choice of locations, evicting another entry. The evicted entry is then placed in its alternate choice of cells, potentially evicting yet another entry. This continues until an open cell is found, or an infinite loop is detected (in which case, two new hash functions are chosen and all entries are deleted and reinserted). It can be shown that as long as the load factor of the table remains below 0.5, then an insertion succeeds in expected constant time. Notice that a search can be performed in *worst-case* constant time, because it can only be stored in one of two possible locations. Give a complete map implementation based on this strategy.

- P-10.66 An interesting strategy for hashing with separate chaining is known as **power-of-two-choices hashing**. Two independent hash functions are computed for each key, and a newly inserted element is placed into the choice of the two indicated buckets that currently has the fewest entries. Give a complete map implementation based on this strategy.
- P-10.67 Implement a `LinkedHashMap` class, as described in Exercise C-10.44, ensuring that the primary map operations run in  $O(1)$  expected time.
- P-10.68 Perform experiments on our `ChainHashMap` and `ProbeHashMap` classes to measure its efficiency using random key sets and varying limits on the load factor (see Exercise R-10.14).
- P-10.69 Perform a comparative analysis that studies the collision rates for various hash codes for character strings, such as polynomial hash codes for different values of the parameter  $a$ . Use a hash table to determine collisions, but only count collisions where different strings map to the same hash code (not if they map to the same location in this hash table). Test these hash codes on text files found on the Internet.
- P-10.70 Perform a comparative analysis as in the previous exercise, but for 10-digit telephone numbers instead of character strings.
- P-10.71 Design a Java class that implements the skip-list data structure. Use this class to create a complete implementation of the sorted map ADT.
- P-10.72 Extend the previous project by providing a graphical animation of the skip-list operations. Visualize how entries move up the skip list during insertions and are linked out of the skip list during removals. Also, in a search operation, visualize the scan-forward and drop-down actions.
- P-10.73 Describe how to use a skip list to implement the array list ADT, so that indexed insertions and removals both run in  $O(\log n)$  expected time.
- P-10.74 Write a spell-checker class that stores a lexicon of words,  $W$ , in a set, and implements a method, `check( $s$ )`, which performs a **spell check** on the string  $s$  with respect to the set of words,  $W$ . If  $s$  is in  $W$ , then the call to `check( $s$ )` returns a list containing only  $s$ , as it is assumed to be spelled correctly in this case. If  $s$  is not in  $W$ , then the call to `check( $s$ )` returns a list of every word in  $W$  that might be a correct spelling of  $s$ . Your program should be able to handle all the common ways that  $s$  might be a misspelling of a word in  $W$ , including swapping adjacent characters in a word, inserting a single character in between two adjacent characters in a word, deleting a single character from a word, and replacing a character in a word with another character. For an extra challenge, consider phonetic substitutions as well.

## Chapter Notes

Hashing is a well-studied technique. The reader interested in further study is encouraged to explore the book by Knuth [61], as well as the book by Vitter and Chen [92]. The denial-of-service vulnerability exploiting the worst-case performance of hash tables was first described by Crosby and Wallach [27], and later demonstrated by Klink and Wälde [58]. The remedy adopted by the OpenJDK team for Java is described in [76].

Skip lists were introduced by Pugh [80]. Our analysis of skip lists is a simplification of a presentation given by Motwani and Raghavan [75]. For a more in-depth analysis of skip lists, please see the various research papers on skip lists that have appeared in the data structures literature [56, 77, 78]. Exercise C-10.42 was contributed by James Lee.